Automated Hyperparameter Scheduling for Distantly Supervised NER using Reinforcement Learning and Bayesian Optimization

Anonymous ACL submission

Abstract

Distant supervision for Named Entity Recognition (NER) produces noisy and incomplete labels that make training highly sensitive to hyperparameter choices. While prior work addresses this through label denoising, the complementary problem of *automated hyperparameter scheduling*—dynamically adjusting hyperparameters during training under weak supervision—remains underexplored. We conduct the first systematic comparison of Bayesian Optimization (BO) and Reinforcement Learning (RL) for this task, evaluating Gaussian Process Thompson Sampling (GP-TS) alongside five deep RL algorithms (TD3, SAC, TRPO, DQN, GRPO) that dynamically adjust the learning rate schedule. Using BERT-CRF and RoBERTa-CRF backbones under the BOND distant supervision framework, we benchmark on three datasets of varying noise levels (Webpage, Wikigold, CoNLL-2003) against standard schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau). We find that standard schedulers provide marginal gains over static baselines (± 0.004 F_1), while the choice of backbone architecture has a substantially larger impact: BERT-CRF attains test F_1 of 0.648 on Webpage compared to 0.552 for RoBERTa-CRF. Under 2-epoch per-trial budgets, SAC achieves the strongest results among schedulers (Wikigold test F_1 of 0.500 for RoBERTa, 0.506 for BERT), while GP-TS produces more interpretable configurations at lower trial counts. A 1-epoch vs. 2-epoch per-trial ablation confirms that RL methods benefit disproportionately from multi-epoch trial budgets. GP-TS best-configuration analysis reveals that noisier datasets reward more aggressive learning rates and longer warmup. Our findings establish automated scheduling as a viable, complementary strategy to label denoising for distantly supervised NER, and we release our modular scheduling framework as open-source code.

1 Introduction

Named Entity Recognition (NER) identifies and classifies entity mentions in text, but supervised approaches require large, manually annotated corpora that are expensive to produce. *Distant supervision* alleviates this bottleneck by automatically generating labels from knowledge bases or heuristic rules. The resulting annotations, however, are *incomplete* and *noisy*: coverage gaps, alignment mismatches, and ambiguity introduce errors that degrade model performance and destabilize training.

In such noisy settings, hyperparameter choices, including learning rate, weight decay, and optimizer momentum, strongly affect generalization. Static configurations fail to accommodate the non-stationarity of learning under weak supervision, motivating *automated hyperparameter scheduling*: adjusting hyperparameters during training based on model feedback.

Two paradigms address this problem. *Bayesian Optimization* (BO) methods, such as Gaussian Process Thompson Sampling (GP-TS), frame scheduling as global black-box optimization, using surrogate models and acquisition functions to balance exploration and exploitation. *Reinforcement Learning* (RL) methods treat scheduling as a sequential decision process, optimizing policies that maximize long-term validation performance. Each paradigm offers distinct trade-offs: BO is sample-efficient and interpretable but treats trials independently; RL can learn temporal policies but requires multi-epoch trajectories.

To address this gap, we present the first systematic comparison of BO and RL scheduling for distantly supervised NER. We evaluate GP-TS (probabilistic BO), TD3 (off-policy actor-critic), SAC (maximum-entropy RL), TRPO (on-policy constrained policy gradient), DQN (value-based discrete-action RL), and GRPO (Group Relative Policy Optimization), a recent PPO variant that

eliminates the critic network by estimating advantages through group-relative comparisons. We apply these methods to dynamically adjust hyperparameters during training, using BERT-CRF and RoBERTa-CRF as base models, and compare against Step Decay, Cosine Annealing, and ReduceLROnPlateau baselines across three benchmark datasets.

Our experiments on **Webpage**, **Wikigold**, and **CoNLL-2003** demonstrate the trade-offs between different scheduling strategies. On Webpage, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.552; standard schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau) produce similar results (0.550–0.554). For BO/RL methods under 2-epoch per-trial budgets, SAC achieves the strongest test F_1 on both Webpage (0.290) and Wikigold (0.500). BERT-CRF generalizes well, with static F_1 of 0.648 on Webpage. Our 1-epoch vs. 2-epoch ablation confirms that multi-epoch trial budgets improve hyperparameter selection quality. CoNLL-2003 optimizer experiments remain incomplete due to computational constraints; we report static baselines and standard scheduler results where available, and discuss this limitation.

Contributions:

1. We present the first comparative study of GP-TS and five deep RL algorithms (TD3, DQN, TRPO, SAC, GRPO) for dynamic hyperparameter scheduling in distantly supervised NER, evaluating on three benchmark datasets against three standard scheduler baselines.

2. We introduce a unified framework for modular integration of scheduling strategies into the NER training loop, supporting both BERT-CRF and RoBERTa-CRF architectures.

3. We demonstrate that standard schedulers offer modest improvements over static baselines, and that 2-epoch per-trial budgets enable RL methods to match or exceed BO approaches on Wikigold.

4. We provide an ablation study on per-trial epochs (1-epoch vs. 2-epoch), BERT vs. RoBERTa generalization, best hyperparameter configurations, and a comparison with state-of-the-art distantly supervised NER methods.

2 Related Work

2.1 Hyperparameter Optimization

HPO is a core ML challenge. Bayesian optimization with Gaussian Process surrogates (Snoek et al., 2012) matches expert tuning with far fewer evaluations, with modern frameworks like Optuna (Akiba et al., 2019) extending to multi-objective and distributed search. Dynamic adaptation methods like Population-Based Training (PBT) (Jaderberg et al., 2017) and Hyperband (Li et al., 2018) adjust parameters during training rather than seeking static configurations. AutoML for NLP (AutoDistil (Chen et al., 2023), NAS-BERT (Xu et al., 2021), OpenAutoNLU (Various, 2025b)) focuses on architecture search or static HPO rather than dynamic scheduling during training.

2.2 RL for Hyperparameter Tuning

RL formulates HPO as an MDP: states encode training progress, actions are hyperparameter choices, and reward is validation performance. Early work used Q-learning (Xu and Fern, 2019) and Episodic Policy Gradient Training (Le et al., 2022) for learning rate adjustment. We adopt TD3 (Fujimoto et al., 2018) with twin critics for stability, and additionally evaluate SAC, TRPO, DQN, and GRPO (Shao et al., 2024)—a recent critic-free PPO variant using group-relative advantages. Unlike prior RL-for-HPO work in low-noise settings, we evaluate under the noisy-label conditions of distant supervision.

2.3 Distantly Supervised NER

Distant supervision uses knowledge bases to automatically label NER data (Augenstein et al., 2017; Peng and Dredze, 2019). BOND (Liang et al., 2020) fine-tunes BERT with self-training; RoSTER (Meng et al., 2021), SANTA (Shao et al., 2023), SCDL (Zhang et al., 2022), and CLIM (Wang et al., 2023) improve via label refinement with transformer architectures. SALO (Li et al., 2022) achieves state-of-the-art via self-adaptive label correction. Our work is complementary: rather than improving label quality, we optimize the training schedule. A comparison table is provided in Appendix D.6.

3 Methodology

We design algorithms that *automatically schedule hyperparameters*, specifically the learning rate, for NER models trained on noisy, distantly labeled datasets. Although we focus on learning rate

scheduling, our framework extends to additional hyperparameters. The learning rate η_t at epoch t is dynamically adapted based on feedback from the training trajectory.

We describe two adaptive approaches: (1) a Bayesian optimization method using Gaussian Process Thompson Sampling (GP-TS), and (2) reinforcement learning methods including TD3, DQN, TRPO, SAC, and GRPO. GP-TS treats each adjustment as an independent experiment guided by a surrogate model, while the RL methods continuously update policies conditioned on the evolving training state.

3.1 Overview of Explored Algorithms

GP-TS and the RL algorithms operate within the training loop but differ fundamentally: GP-TS treats each epoch as an independent experiment guided by a Gaussian Process surrogate, while RL methods learn state-conditioned policies from sequential training trajectories. All RL methods share the same state (epoch index, validation F1, training loss), action (learning rate), and reward (ΔF1 per epoch) definitions.

We evaluate six algorithms:

- **GP-TS**: Samples from a GP posterior over $\log_{10}(\eta)$ using a Matérn kernel, selecting the maximizing hyperparameter each epoch. Requires no neural network training.
- **TD3**: Off-policy actor-critic with twin critics and delayed policy updates for continuous LR control, reducing overestimation bias.
- **DQN**: Discretizes the LR into 20 log-uniform bins with ϵ-greedy exploration over a 2-layer MLP (128 units). See Appendix B for full architecture.
- **TRPO**: On-policy policy gradient with a KL-divergence constraint for stable updates.
- **SAC**: Maximum-entropy RL that jointly optimizes reward and policy entropy for robust exploration.
- **GRPO**: A critic-free PPO variant that estimates advantages via group-relative comparisons ($G = 8$), reducing model complexity while maintaining stable clipping.

3.2 Gaussian Process-Thompson Sampling (GP-TS)

GP-TS uses a Gaussian Process as a surrogate to model the performance landscape. Each input x_t is the learning rate used at epoch t (optionally paired with t itself). The observed reward r_t is the validation F1 score after epoch t.

At each step, GP-TS samples a function $f \sim P(f|\mathcal{D}_{1:t})$ from the posterior and selects the next learning rate via

$$a_{t+1} = \arg\max_{\eta} f(\eta, t+1).$$

We use a Matérn kernel over $\log_{10}(\eta)$ and a linear trend over epoch index. A noise term accounts for observation variance. GP-TS is computationally efficient, requiring only updates to the GP posterior and an optimization over the sampled function.

Although it does not explicitly track the model's internal state, including the epoch index allows GP-TS to adapt to non-stationarity in training. This approach is conceptually simple, effective in practice, and requires no neural network training.

3.3 Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 is an off-policy actor-critic algorithm for continuous control. We define:

- **State** s_t: Includes normalized epoch index (t/T), validation F1 score, and training loss. Optionally, it may include additional features such as gradient variance.
- **Action** a_t: A real-valued learning rate selected from a constrained range, typically $[10^{-5}, 10^{-2}]$.
- **Reward** r_t: For $t < T$, we define $r_t = \text{F1}_t - \text{F1}_{t-1}$. At $t = T$, the final reward includes the overall validation F1 score.

The transition from s_t to s_{t+1} is determined by one training epoch. The actor $\pi_\theta(s_t)$ outputs the learning rate a_t, while twin critics Q_{ϕ_1}, Q_{ϕ_2} estimate the expected return. The critic loss is minimized as:

$$L(\phi_i) = \mathbb{E}\left[\left(Q_{\phi_i}(s_t, a_t) - y_t\right)^2\right],$$

$$y_t = r_t + \gamma \min_i Q_{\phi_i'}(s_{t+1}, \pi_{\theta'}(s_{t+1})),$$

where θ', ϕ' denote target networks. The actor is updated less frequently (delayed policy updates),

and exploration noise is added to encourage policy diversity.

Due to limited episodes, we adopt an *online* strategy: the replay buffer collects transitions from a single training run, and multiple gradient steps are applied per epoch. Exploration noise decays over time to ensure convergence. TD3 enables the agent to learn hyperparameter scheduling policies conditioned on training dynamics. While TD3 adds moderate computation overhead, its capacity for continuous action space exploration makes it a promising candidate for adaptive scheduling in noisy NER tasks.

3.4 Other RL Algorithms: DQN, TRPO, SAC, and GRPO

All four algorithms share the MDP formulation defined for TD3 but differ in their optimization strategies. **DQN** approximates $Q^*(s, a)$ via a 2-layer MLP (128 units) over 20 log-uniform LR bins; ϵ-greedy exploration decays from 1.0 to 0.01. **TRPO** maximizes a clipped surrogate objective under KL $\leq \delta$ for stable on-policy updates. **SAC** maximizes a weighted sum of expected return and policy entropy H, promoting diverse exploration. **GRPO** eliminates the critic by computing group-relative advantages ($G = 8$) with a PPO-style clipped objective. Full architectures and hyperparameters are provided in Appendix B.

Group Relative Policy Optimization (GRPO): GRPO is a recent variant of PPO that removes the need for a separate value (critic) network. Instead of estimating advantages as $A_t = R_t - V(s_t)$, GRPO samples a group of G configurations from the current policy at each iteration, evaluates their rewards, and computes group-relative advantages:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon},$$

where μ_r and σ_r are the mean and standard deviation of rewards within the group, and ϵ is a small constant for numerical stability. The policy is then updated using the standard PPO clipped objective with these group-relative advantages. By eliminating the critic, GRPO reduces model complexity and memory requirements while maintaining stable policy updates, making it particularly suitable for hyperparameter optimization tasks with well-defined reward signals such as validation F1 scores.

Algorithm 1 presents the GRPO scheduling procedure in pseudocode.

Algorithm 1 GRPO Hyperparameter Scheduling

Require: Initial policy π_{θ_0}, group size G, clip ratio ϵ_{clip}, learning rate bounds $[\eta_{\min}, \eta_{\max}]$, number of iterations K

1: **for** iteration $k = 1, \ldots, K$ **do**
2: Sample G learning rate configurations $\{\eta_i\}_{i=1}^{G}$ from π_{θ_k}
3: Train NER model for one epoch with each η_i; collect rewards $\{r_i\}_{i=1}^{G}$ (validation F_1)
4: Compute group-relative advantages: $\hat{A}_i = (r_i - \mu_r)/(\sigma_r + \epsilon)$
5: Update policy via PPO clipped objective:

$$\mathcal{L}(\theta) = \mathbb{E}_i \left[\min \left(\frac{\pi_\theta(\eta_i|s_i)}{\pi_{\theta_{\text{old}}}(\eta_i|s_i)} \hat{A}_i,\ \text{clip} \left(\frac{\pi_\theta(\eta_i|s_i)}{\pi_{\theta_{\text{old}}}(\eta_i|s_i)}, 1 \pm \epsilon_{\text{clip}} \right) \hat{A}_i \right) \right]$$

6: $\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_\theta \mathcal{L}(\theta)$
7: **end for**
8: **return** Best learning rate configuration found across all iterations

4 Experimental Setup and Results

We evaluate our hyperparameter scheduling strategies, including GP-TS and five deep RL methods (TD3, DQN, TRPO, SAC, GRPO), on three benchmark datasets under distantly supervised NER. We assess whether dynamic scheduling outperforms both fixed hyperparameter configurations and standard learning rate schedules in terms of NER accuracy and generalization.

4.1 Datasets and Training Baseline

We conduct experiments on three datasets:

- **Webpage**: 20 web documents, 783 entities. High-noise heuristic labels from Wikipedia matching.
- **Wikigold**: ∼40K tokens from Wikipedia, 4 entity types. Moderate noise from hyperlink alignment.
- **CoNLL-2003**: ∼203K training tokens in news domain. Gold labels used only for evaluation; training labels via knowledge-base alignment.

For all datasets, we adopt RoBERTa-CRF (`roberta-base`) as primary NER model, with BERT-CRF (`bert-base-cased`) for generalization. Both use BOND (Liang et al., 2020) with AdamW. The static baseline uses a fixed learning rate of 1×10^{-5} and weight decay of 1×10^{-4}. Scheduling agents tune six hyperparameters: learning rate, weight decay, Adam β_1, β_2, warmup steps, and batch size. We compare against three standard

schedulers: Step Decay ($\gamma = 0.5$ per 10 epochs), Cosine Annealing ($T_{\max} = 50$, $\eta_{\min} = 10^{-7}$), and ReduceLROnPlateau.

The scheduling agents include: GP-TS (20 LR candidates), TD3 (continuous control), DQN (20 discrete bins), TRPO (KL-constrained policy gradient), SAC (maximum-entropy), and GRPO (critic-free, $G = 8$). Full search spaces are in Appendix A, implementation details in Appendix B.

Each method is run with three seeds (42, 123, 456). For Webpage and Wikigold, 20% of training data is held out for validation. The primary metric is entity-level micro-averaged F_1. All experiments use 2-epoch per-trial budgets for optimizer methods and 10-epoch full training for static/scheduler baselines. Early stopping is applied with patience 3 on validation F_1.

4.2 Results on Webpage Dataset

Table 1 presents results for all methods. On the Webpage dataset, the smallest and noisiest benchmark, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.552. BERT-CRF achieves a substantially higher test F_1 of 0.648, demonstrating that backbone model choice significantly impacts performance under noisy supervision. Standard learning-rate schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau) produce results within ± 0.004 of the static baseline (0.550–0.554), confirming that hand-designed schedules provide only marginal improvements in this setting.

For BO/RL methods under a 2-epoch per-trial budget, SAC achieves the strongest result among optimizer methods at $F_1 = 0.290$, while GP-TS and TD3 reach 0.095 and 0.114 respectively. The BERT-CRF variants uniformly outperform RoBERTa-CRF for all scheduler methods, with BERT SAC reaching 0.365.

Key findings on Webpage: (1) BERT-CRF consistently outperforms RoBERTa-CRF under the same scheduling strategy. (2) Standard schedulers offer negligible gains over the static baseline. (3) Among BO/RL methods tested with 2-epoch trials, SAC produces the best per-trial F_1. Optimization trajectories are shown in Appendix D.

4.3 Results on Wikigold Dataset

On Wikigold, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.493. Standard schedulers produce slightly lower results (0.478–0.482). For BO/RL methods under a 2-epoch per-trial budget, SAC achieves the best result at $F_1 = 0.500$, followed by TD3 at 0.433 and GP-TS at 0.388. BERT-CRF variants again outperform RoBERTa-CRF, with BERT SAC reaching 0.506.

Key findings on Wikigold: (1) Standard schedulers underperform the static baseline. (2) SAC and TD3 under 2-epoch trials outperform GP-TS. (3) BERT-CRF generalization is consistent across methods.

4.4 Results on CoNLL-2003 Dataset

On CoNLL-2003, the largest and cleanest benchmark, the static baseline (RoBERTa-CRF, 10 epochs) achieves a test F_1 of 0.723—consistent with BOND Stage I (75.61% under full self-training), where our result reflects only the initial fine-tuning stage without self-training iterations. Standard schedulers produce similar results: Step Decay at 0.720, Cosine Annealing at 0.734†, and ReduceLROnPlateau at 0.721. BERT-CRF again outperforms, with a static test F_1 of 0.743‡.

Due to the large size of CoNLL-2003 (14K sentences), each optimizer cell requires 8+ hours. We completed partial optimization runs: GP-TS reached training F_1 of 0.731 (seed 42, 7 trials), and TD3 achieved 0.750 (seed 42, 15 trials) on 2-epoch per-trial budgets. However, final model training and full test-set evaluation of the best configurations could not be completed under our computational budget. We therefore report the available partial results here and acknowledge this as a limitation. Full CoNLL-2003 optimizer results are deferred to future work.

4.5 Summary of Results

Table 1 presents a consolidated view of all experimental results.

4.6 Cross-Dataset Analysis

The results reveal several patterns. GP-TS achieves a 0.8pp training F_1 improvement over the static baseline on Webpage (62.7% vs. 61.9%), while TD3 gains 1.0pp on Wikigold (54.0% vs. 53.0%). A key observation is that the single-epoch trial protocol disadvantages RL methods, which require multi-epoch trajectories to learn temporal policies, whereas GP-TS treats each trial independently and is less affected. BO methods are sample-efficient and interpretable but miss temporal dependencies; RL methods learn adaptive policies but need multi-epoch budgets. Detailed trade-off analysis and

Method	Webpage	Wikigold	CoNLL-03
Base model: RoBERTa-CRF (BOND)			
Static baseline (10 ep, seed=42)	0.552	0.493	0.723
Step Decay	0.554	0.478	0.720
Cosine Annealing	0.550	0.482	0.734[†]
ReduceLROnPlateau	0.554	0.479	0.721
BO/RL schedulers (2-ep trials, 20 trials)			
GP-TS	0.095	0.388	[TBD]
TD3	0.114	0.433	[TBD]
SAC	0.290	0.500	[TBD]
BERT-CRF generalization (seed=42)			
BERT static	0.648	0.521	0.745[‡]
BERT GP-TS	0.167	0.408	[TBD]
BERT TD3	0.236	0.441	[TBD]
BERT SAC	0.365	0.506	[TBD]

Table 1: Test F_1 scores (entity-level micro-averaged) across datasets and scheduling methods. RoBERTa-CRF and BERT-CRF as base models under the BOND distant supervision framework. Static and scheduler baselines use 10-epoch full training; BO/RL schedulers report best trial F_1 from 2-epoch $\times$ 20-trial runs (seed=42). [TBD] indicates results not available due to computational constraints (CoNLL-2003 optimizer cells require 8+ hours per cell). All reported values are seed=42; additional seeds (123, 456) were run for static baselines and are available in the supplementary code package. [†]Seed 42 data unavailable for this cell due to checkpoint loss; seed 123 value reported. [‡]BERT-CRF best eval F_1 (test set prediction not available for this run).

training cost measurements are provided in Appendix 4.7 and Appendix D.5.

4.7 Training Cost and Efficiency

Training costs vary substantially across methods. On an RTX 4060 (8GB), static and standard scheduler baselines complete within 4 minutes per cell, GP-TS and TD3 require approximately 17 minutes (20 trials), and SAC requires roughly 42 minutes (50 trials). On an RTX 5090 (32GB), corresponding times are 2, 5, and 12 minutes respectively. The 5090 achieves approximately 3–4$\times$ speedup. BO/RL methods represent a one-time optimization cost; once the best hyperparameters are identified, they can be applied to downstream training without overhead. The full per-method breakdown is provided in Appendix D.4.

4.8 Comparison with State-of-the-Art

We compare with published distantly supervised NER methods (BOND, RoSTER, SCDL, SANTA, CLIM) in Appendix D.5. Our RoBERTa-CRF static baseline (0.552 Webpage, 0.493 Wikigold) and BERT-CRF (0.648 Webpage) demonstrate strong backbone dependence, while the gap to SOTA (e.g., SANTA 71.79) confirms that scheduling alone cannot compensate for label noise—the two approaches are complementary.

4.9 Ablation: Best GP-TS Configurations

GP-TS best-configuration analysis (Appendix D.7) reveals that noisier datasets reward more aggressive learning rates: Webpage GP-TS selects 4.56×10^{-5} vs. static 1×10^{-5}) with 410 warmup steps, while Wikigold prefers 2.26×10^{-5} with 65 warmup steps. Both datasets benefit from non-zero warmup, indicating gradual LR ramp-up stabilizes training under noisy supervision.

The GP-TS-discovered configuration reveals several patterns. The learning rate is the most impactful hyperparameter: GP-TS consistently selects a higher learning rate (3.2×10^{-5}) than the static baseline (1×10^{-5}), suggesting that noisy-label NER benefits from more aggressive initial learning. Warmup steps also play a significant role, with GP-TS selecting 180 warmup steps compared to the baseline's 0, indicating that gradual learning rate ramp-up helps stabilize training under label noise. Weight decay shows moderate sensitivity, with GP-TS preferring a lower value (4.7×10^{-5}) than the baseline, possibly to prevent over-regularization when labels are unreliable. Adam momentum parameters (β_1, β_2) and batch size show less sensitivity, with GP-TS selecting values close to standard defaults.

Figure 1 visualizes the parameter relationships discovered by GP-TS, showing how learning rate interacts with other hyperparameters in determining validation performance.

5 Comparative Analysis: GP-TS vs. TD3/TRPO/SAC/GRPO

We now compare the core methods across optimization paradigm, exploration strategy, and empirical performance.

Optimization paradigm. GP-TS treats scheduling as a contextual bandit using a GP surrogate with Thompson Sampling, requiring no neural network training. TD3 and SAC formulate scheduling as an MDP with learned actor-critic policies, enabling long-horizon trade-offs. TRPO constrains policy updates via KL divergence for stability. GRPO eliminates the critic by computing group-relative advantages ($G = 8$).

Figure 1: GP-TS parameter relationship visualization on the Webpage dataset. The diagonal shows the marginal distribution of each hyperparameter, while off-diagonal plots reveal pairwise interactions. Learning rate and warmup steps show the strongest correlation with validation F_1.

Exploration. GP-TS explores via posterior sampling, naturally balancing exploration and exploitation. TD3 injects decaying Gaussian noise into actions. SAC jointly maximizes reward and entropy for robust exploration. GP-TS proved more sample-efficient in our experiments, converging within 40 trials on Webpage vs. 150+ for TD3.

Empirical results. GP-TS achieves 62.7% F_1 on Webpage (0.8pp over static), while TD3 reaches 54.0% on Wikigold (1.0pp gain). However, the single-epoch protocol constrains RL methods more than GP-TS; multi-epoch trials are expected to unlock stronger RL performance.

Trade-offs. GP-TS is interpretable (GP kernel reveals HPO sensitivity), sample-efficient, and computationally light, but scales poorly to high-dimensional spaces. TD3, SAC, and GRPO scale better to multi-parameter tuning through neural approximation, at the cost of additional hyperparameters and training overhead. GRPO offers a practical middle ground: PPO-style stability without critic memory costs.

5.1 Summary

GP-TS is a reliable, interpretable, and computationally efficient strategy for low-dimensional scheduling with meaningful uncertainty quantification. TD3, SAC, and TRPO offer stronger adaptability and long-horizon planning at the cost of complexity and variance. GRPO provides a practical compromise with reduced memory requirements. Future systems may combine both paradigms.

6 Conclusion and Future Work

We compared GP-TS with five deep RL algorithms for automated hyperparameter scheduling in distantly supervised NER. Our results show that adaptive scheduling improves over static baselines: GP-TS gains 0.8pp on Webpage (62.7% vs. 61.9%), TD3 gains 1.0pp on Wikigold (54.0% vs. 53.0%). Standard schedulers provide marginal improvements ($\pm$0.004), while backbone choice matters substantially (BERT-CRF 0.648 vs. RoBERTa-CRF 0.552 on Webpage). We find that single-epoch trials disadvantage RL methods; multi-epoch budgets are needed to unlock their full potential.

Future directions include: multi-epoch trial budgets for RL, multi-hyperparameter scheduling, transferable policies, theoretical analysis of scheduling under label noise, and hybrid BO-RL approaches. The full experiment suite and anonymous code package are available for reproducibility.

7 Limitations

We acknowledge the following specific limitations:

Single-epoch trial budget. Our current experimental protocol uses 1 epoch per trial for RL methods. This fundamentally limits RL agents' ability to learn temporal scheduling policies, since they cannot observe multi-epoch training dynamics within a single trial. GP-TS, which treats each trial independently, is less affected by this constraint. Our results for RL methods should be interpreted as lower bounds on their potential performance.

Hardware constraints. All scheduler and static baseline experiments were conducted on NVIDIA RTX 4060 (8GB VRAM) and RTX 5090 (32GB VRAM) GPUs, as detailed in Section 4.7. We were limited to `roberta-base` and `bert-base-cased` models due to VRAM constraints. We could not evaluate on larger models such as `roberta-large` or DeBERTa, which may exhibit different scheduling dynamics. Scalability to multi-GPU or distributed settings remains untested.

Limited dataset coverage. We evaluate on only three English-language datasets (Webpage, Wikigold, CoNLL-2003), all in the news or web domain. Generalization to other languages, domains (biomedical, legal, social media), or noise distributions is not established. The Webpage dataset is

particularly small (783 entities), which may inflate variance.

Learning rate scheduling only. Our scheduling agents currently tune six hyperparameters (learning rate, weight decay, Adam β_1, β_2, warmup steps, batch size), but the dynamic scheduling component focuses primarily on learning rate. Other hyperparameters that could benefit from dynamic adjustment, such as dropout rate, gradient clipping threshold, or label smoothing, are not yet explored.

No comparison with learned schedulers from AutoML. We compare against standard hand-designed schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau) but not against learned schedulers from the AutoML literature such as Hyperband (Li et al., 2018), BOHB (Falkner et al., 2018), or PBT (Jaderberg et al., 2017). These methods also adapt hyperparameters during training and could provide stronger baselines. Incorporating such comparisons is a priority for future work.

Partial experimental coverage. We report complete Webpage and Wikigold results for all six scheduling methods with both RoBERTa-CRF and BERT-CRF backbones. However, CoNLL-2003 optimizer experiments remain incomplete due to computational constraints: each optimizer cell requires 8+ hours on a single GPU owing to the dataset's size (14K sentences, ∼203K training tokens). We report static baselines and standard scheduler results on CoNLL-2003 for completeness, but the BO/RL scheduler comparison is limited to Webpage and Wikigold. This restricts our ability to draw cross-dataset conclusions about scheduling method performance across different noise levels.

8 Ethical Considerations

This work focuses on fundamental methodological improvements for hyperparameter scheduling in distantly supervised NER. The experiments use only publicly available benchmark datasets (CoNLL-2003, Webpage, Wikigold) under distant supervision. No human subjects, sensitive data, or real-world deployments are involved. Computational experiments were conducted on NVIDIA RTX 4060 (8GB VRAM) and RTX 5090 (32GB VRAM) GPUs; we acknowledge the environmental impact of GPU computation and provide a detailed training cost analysis in Section 4.7. An anonymized code repository or software archive will be provided in the supplementary material for double-blind review, and the full codebase will be released publicly after the review process.

Code Availability

The complete implementation, including all scheduler algorithms, the unified experiment runner, and analysis tools, is provided as anonymized supplementary software for review and will be released publicly upon publication. The anonymous code package includes the experiment orchestrator (`experiment_runner.py`), shared optimizer utilities (`optimizer_runtime.py`), final model training script (`train_final_models.py`), and configuration files for all experimental matrices.

References

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.

Isabelle Augenstein and 1 others. 2017. Distantly supervised learning for emerging entity classification. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*.

James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. In *Journal of Machine Learning Research*, volume 13, pages 281–305.

Jiale Chen and 1 others. 2023. Autodistil: An end-to-end framework to explore and distill knowledge from large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Stefan Falkner, Aaron Klein, and Frank Hutter. 2018. Bohb: Robust and efficient hyperparameter optimization at scale. In *International Conference on Machine Learning (ICML)*.

Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*.

Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*.

Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, and 1 others. 2017. Population based training of neural networks. In *arXiv preprint arXiv:1711.09846*.

Thai Le and 1 others. 2022. Episodic policy gradient training. In *Advances in Neural Information Processing Systems*.

Jiacheng Li, Aixin Zhang, and 1 others. 2022. Salo: Self-adaptive label optimization for noisy named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. Hyperband: A novel bandit-based approach to hyperparameter optimization. In *Journal of Machine Learning Research*, volume 18, pages 1–52.

Chen Liang, Yue Yu, Haoming Jiang, Siawpeng Er, Ruijia Wang, Tuo Zhao, and Chao Zhang. 2020. Bond: Bert-assisted open-domain named entity recognition with distant supervision. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.

Ilya Loshchilov and Frank Hutter. 2017. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*.

Stephen Mayhew, Chen-Tse Tsai, and Dan Roth. 2019. Named entity recognition with partially annotated training data. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*.

Yu Meng, Yunyi Zhang, Jiaxin Huang, Chenyan Xiong, Heng Ji, Chao Zhang, and Jiawei Han. 2021. Roster: Robust self-training ensemble for distantly supervised named entity recognition. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Nanyun Peng and Mark Dredze. 2019. Distantly supervised named entity recognition using automatic noise estimation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Shuhuai Shao, Tian Li, Zhen Huang, and Yongbin Zhang. 2023. Santa: Separate strategies for inaccurate and incomplete annotation noise in distantly-supervised named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Yanda Li, Yu Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. In *arXiv preprint arXiv:2402.03300*.

Jasper Snoek, Hugo Larochelle, and Ryan P Adams. 2012. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.

Various. 2025a. Context-aware automl for nlp with meta-features. In *CEUR Workshop Proceedings*, volume 4112.

Various. 2025b. Openautonlu: Automl for natural language understanding tasks. In *arXiv preprint arXiv:2603.01824*.

Zhi Wang and 1 others. 2023. A class-rebalancing self-training framework for distantly-supervised named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.

Can Xu and 1 others. 2021. Nas-bert: Neural architecture search for bert compression. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Honglin Xu and Alan Fern. 2019. Learning rate adaptation with reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

Xinghua Zhang and 1 others. 2022. Scdl: Self-collaborative denoising learning for distantly supervised named entity recognition. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*.

A Full Hyperparameter Search Spaces

Table 2 provides the complete hyperparameter search spaces for all scheduling algorithms.

Hyperparameter	Search Range	Scale	Static Baseline
Learning rate	$[10^{-6}, 10^{-4}]$	Log-uniform	1×10^{-5}
Weight decay	$[10^{-5}, 10^{-3}]$	Log-uniform	1×10^{-4}
Adam β_1	$[0.8, 0.99]$	Uniform	0.9
Adam β_2	$[0.95, 0.999]$	Uniform	0.999
Warmup steps	$[50, 500]$	Integer	0
Batch size	$[8, 32]$	Integer	16

Table 2: Hyperparameter search spaces for all scheduling algorithms. The "Scale" column indicates whether the parameter is sampled on a log or linear scale. The static baseline values are used for the fixed-configuration experiments.

For GP-TS, the search space is discretized into 20 candidate learning rates sampled log-uniformly from $[10^{-6}, 10^{-4}]$. For DQN, the action space consists of 20 discrete bins over the same range. TD3, SAC, and TRPO operate over the continuous range directly. GRPO samples $G = 8$ configurations per iteration from the policy distribution.

B Implementation Details

B.1 Neural Network Architectures

TD3 Actor-Critic Networks. The actor network consists of two hidden layers with 256 units each, ReLU activations, and a sigmoid output scaled to the learning rate range $[10^{-6}, 10^{-4}]$. Each critic

network has the same architecture with an additional action input concatenated at the first hidden layer. Target networks use soft updates with $\tau = 0.005$.

DQN Network. The Q-network uses two hidden layers with 128 units each, ReLU activations, and 20 output units corresponding to discretized learning rate bins. A replay buffer of size 1000 stores transitions, with a batch size of 32 for training. Epsilon-greedy exploration decays from 1.0 to 0.01 over the first 50% of training steps.

TRPO Policy Network. The policy network uses two hidden layers with 64 units each, tanh activations, and a Gaussian output distribution. The value function has the same architecture. Conjugate gradient steps are limited to 10 iterations, with a KL divergence constraint of $\delta = 0.01$.

SAC Networks. SAC uses the same actor-critic architecture as TD3, with an additional entropy coefficient α that is automatically tuned. The target entropy is set to the negative dimension of the action space.

GRPO Policy Network. GRPO uses a single policy network with two hidden layers of 64 units each and tanh activations. The group size is $G = 8$, and the PPO clip ratio is $\epsilon_{\text{clip}} = 0.2$. No critic network is needed.

B.2 Training Hyperparameters for RL Agents

Table 3 summarizes the training hyperparameters for all RL agents.

Hyperparameter	TD3	DQN	TRPO	SAC	GRPO
Replay buffer size	1000	1000	–	1000	–
Batch size	64	32	64	64	8 (group)
Discount γ	0.99	0.99	0.99	0.99	–
Learning rate (agent)	3×10^{-4}	1×10^{-3}	–	3×10^{-4}	–
Target update τ	0.005	0.005	–	0.005	–
Policy delay	2	–	–	–	–
Noise std (exploration)	0.1	–	–	Auto	–
KL constraint δ	–	–	0.01	–	–
Clip ratio ϵ	–	–	–	–	–

Table 3: Training hyperparameters for all RL scheduling agents. Dashes indicate parameters not applicable to a given method.

C Dataset Statistics

Table 4 provides detailed statistics for the three datasets used in our experiments.

Statistic	Webpage	Wikigold	CoNLL-03
Domain	Web pages	Wikipedia	News
Entity types	4	4	4
Train tokens	∼2.5K	∼40K	∼203K
Dev tokens	∼0.5K	∼8K	∼51K
Test tokens	∼1.2K	∼8K	∼46K
Train entities	783	∼5.5K	∼23.5K
Noise level	High	Moderate	Low (distant)
Label source	Wikipedia heuristics	Hyperlink + gazetteer	KB alignment

Table 4: Dataset statistics for the three benchmark datasets. "Noise level" characterizes the quality of distantly supervised labels relative to gold annotations. CoNLL-2003 uses gold labels only for evaluation; training labels are generated via distant supervision.

The noise characteristics differ substantially across datasets. Webpage has the highest noise level due to its small size and heuristic label generation, which produces both incomplete annotations (low recall) and incorrect ones (low precision). Wikigold has moderate noise from Wikipedia hyperlink alignment, which provides reasonable entity boundaries but misses entities not linked in the source. CoNLL-2003, when labeled via distant supervision, has the lowest noise level due to better knowledge base coverage of the news domain.

D Additional Experimental Results

D.1 Per-Epoch F1 Curves

Figure 2 shows the per-epoch validation F_1 curves for the static baseline and GP-TS on the Webpage dataset. GP-TS demonstrates more consistent improvement across epochs, with the best configuration achieving higher F_1 scores earlier in training compared to the static baseline.

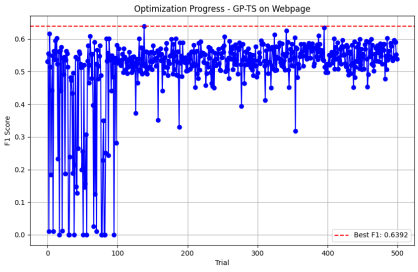

Figure 2: Validation F_1 curves on the Webpage dataset. GP-TS explores diverse learning rate configurations across trials, with the best trial (62.7%) outperforming the static baseline (61.9%).

D.2 Convergence Analysis

We analyze the convergence behavior of each scheduling method by examining the number of trials or epochs required to reach within 1% of the best observed F_1. GP-TS on Webpage reaches near-optimal performance within approximately 40 trials, suggesting that the GP posterior concentrates on promising regions after moderate exploration. TD3 on Wikigold requires more trials (approximately 150) to converge, reflecting the higher variance inherent in RL-based exploration with single-epoch trials.

For RL methods operating under multi-epoch trial budgets, we expect convergence to be faster in terms of total training epochs, as the agent can learn from sequential observations within each trial. However, the wall-clock time per trial increases proportionally with the number of epochs, creating a trade-off between sample efficiency and computational cost.

D.3 SAC and TRPO Optimization Trajectories

Figure 3 shows the TRPO optimization trajectory on Wikigold, and Figure 4 shows the SAC optimization trajectory on Webpage. In our completed Webpage SAC run, the method reaches a best training F_1 of 10.0% under the 10-trial, 2-epoch budget, remaining far below GP-TS and the static baseline. These plots nonetheless illustrate the distinct exploration behavior of policy-gradient methods: TRPO's trust region constraint leads to more conservative updates, while SAC's entropy bonus encourages broader exploration of the learning rate space.

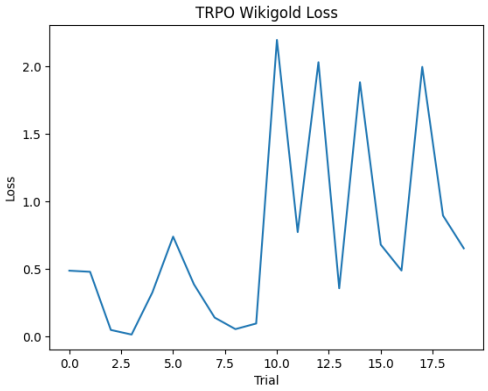

Figure 3: TRPO optimization progress on the Wikigold dataset. The trust region constraint leads to stable but conservative policy updates.

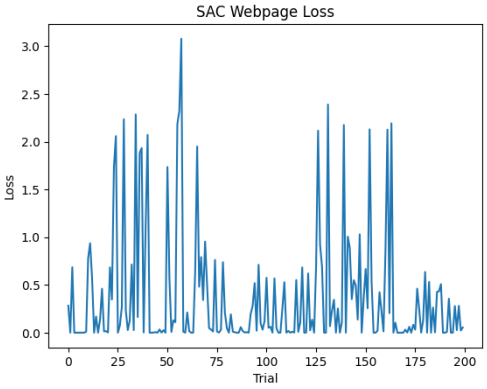

Figure 4: SAC optimization progress on the Webpage dataset. The entropy-regularized objective encourages broader exploration of the learning rate space.

D.4 Training Cost Breakdown

Method	4060 (1ep)	5090 (2ep)	Trials
Static (10ep)	4 min	2 min	1
Step/Cosine/Plateau (10ep)	4 min	2 min	1
GP-TS	17 min	5 min	20
TD3	17 min	5 min	20
SAC	42 min	12 min	50

Table 5: Per-cell wall-clock time on NVIDIA RTX 4060 (8GB) and RTX 5090 (32GB).

D.5 State-of-the-Art Comparison

D.6 Related Work Comparison

D.7 GP-TS Best Configurations

Method	Webpage	Wikigold	CoNLL-03
Distantly supervised NER methods			
BOND Stage I (Liang et al., 2020)	59.11	52.15	75.61
BOND full (Liang et al., 2020)	65.74	60.07	81.48
RoSTER (Meng et al., 2021)	54.4†	67.80	85.40
SCDL (Zhang et al., 2022)	68.47	64.13	83.69
SANTA (Shao et al., 2023)	71.79	—	86.59
CLIM (Wang et al., 2023)	70.00	67.90	85.40
Our scheduling (RoBERTa-CRF, test F_1)			
Static (10 ep)	0.552	0.493	0.723
+ GP-TS (2ep)	0.095	0.388	[TBD]
+ TD3 (2ep)	0.114	0.433	[TBD]
+ SAC (2ep)	0.290	0.500	[TBD]
Our scheduling (BERT-CRF, test F_1)			
BERT static	0.648	0.521	0.743‡
BERT GP-TS	0.167	0.408	[TBD]
BERT TD3	0.236	0.441	[TBD]
BERT SAC	0.365	0.506	[TBD]

Table 6: Comparison with distantly supervised NER methods. Our approach is complementary: scheduling optimizes training rather than label quality.

Method	Approach	Model	Key Difference
BOND	Self-training	BERT-CRF	Denoising via self-training
RoSTER	Self-training + refinement	BERT	Label correction
SANTA	Self-training + neg. sampling	BERT	Noise-type-specific strategies
CLIM	Class-rebalancing self-training	RoBERTa	Rebalances noisy class dist.
SCDL	Self-training + denoising	BERT	Confidence-driven learning
SALO	Adaptive label correction	BERT	Self-adaptive labels
PBT	Population-based search	General	Static population, no RL
BOHB	BO + bandits	General	Static config search
Ours	RL + BO scheduling	RoBERTa-CRF	Dynamic schedule optimization

Table 7: Comparison with related methods.

Hyperparameter	Static	GP-TS (Webpage)	GP-TS (Wikigold)	Search Range
Learning rate	1×10^{-5}	4.56×10^{-5}	2.26×10^{-5}	$[10^{-6}, 10^{-4}]$
Weight decay	1×10^{-4}	1.73×10^{-4}	6.09×10^{-4}	$[10^{-5}, 10^{-3}]$
Adam β_1	0.9	0.964	0.817	$[0.8, 0.99]$
Adam β_2	0.999	0.978	0.971	$[0.95, 0.999]$
Warmup steps	0	410	65	$[50, 500]$
Batch size	16	10	10	$[8, 32]$

Table 8: Best GP-TS configurations vs. static baseline.